

/start 18:05 ✓✓

Привет! Чему бы вы хотели научиться?

This message was sent automatically with n8n 18:05

Мне интересна тема QA 18:05 ✓✓

Какой у вас сейчас уровень знаний в области QA?
(начальный/средний/продвинутый)

This message was sent automatically with n8n 18:05

начальный 18:05 ✓✓

Вот ваш контент-план для изучения основ QA на начальном уровне:

Тема 1: Основы тестирования ПО

- Цели и задачи тестирования
- Типы тестирования (функциональное, нефункциональное)
- Этапы жизненного цикла тестирования

Тема 2: Инструменты и техники тестирования

- Тест-кейсы и чек-листы
- Ручное тестирование vs автоматическое тестирование
- Отчетность о дефектах и их классификация

Тема 3: Практика проведения тестирования

- Проведение регрессионного тестирования
- Обзор популярных инструментов автоматизации
- Основные метрики качества продукта

Этот план сохранен в моей памяти. Подтвердите готовность начать обучение.

This message was sent automatically with n8n 18:05



Готова 18:05 ✓✓

📌 Понятие: Цели и задачи тестирования

✅ Объяснение простыми словами:

Представьте, вы построили дом, но не проверили прочность фундамента, качество материалов и соответствие проекту. Такой дом может рухнуть при первом сильном ветре! Так и с программами: если их не протестировать, могут возникнуть серьезные проблемы после запуска. Цель тестирования — убедиться, что программное обеспечение сделано качественно и надежно, а также вовремя обнаружить и исправить возможные дефекты до того, как ими воспользуется конечный пользователь.

⚙️ Практическое применение:

- Обеспечение качества: проверка, что программа полностью соответствует заданным требованиям и документации.
- Идентификация дефектов: поиск ошибок и проблемных мест перед запуском продукта.
- Снижение расходов: ранняя фиксация ошибок позволяет избежать больших затрат на их устранение позже.

🔍 Ключевые аспекты:

- Раннее начало тестирования сокращает затраты времени и денег.
- Тестирование включает анализ как самого кода, так и поведения программы под нагрузкой.
- Оно подтверждает, что продукт соответствует ожиданиям пользователей и стандартам качества.

💡 Пример из практики:

Допустим, компания разрабатывает мобильное приложение для заказа еды. Перед релизом команда проводит тестирование, проверяя работу каждого элемента интерфейса, стабильность работы сервера и удобство навигации. Это позволяет выявить баги и улучшить функционал еще до выхода приложения на рынок.

приложения на рынок.

Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:05



да 18:05 ✓✓

Типы тестирования (функциональное, нефункциональное)

Понятие:

В процессе разработки программного обеспечения используются разные подходы к проверке качества продукта. Основные типы тестирования делятся на две группы: **функциональное** и **нефункциональное**.

Объяснение простыми словами:

Представьте себе автомобильную мастерскую. Функциональное тестирование похоже на проверку всех деталей машины отдельно друг от друга, например, зажигания, тормозов и фар. Оно помогает убедиться, что каждая деталь работает правильно сама по себе. Нефункциональное же тестирование больше напоминает общую оценку автомобиля после сборки: насколько он устойчив на дороге, удобно ли им управлять, безопасна ли поездка?

Таким образом, функциональное тестирование проверяет каждую отдельную функцию программы, тогда как нефункциональное оценивает такие характеристики, как быстродействие, безопасность и удобство интерфейса.

Практическое применение:

Вот несколько примеров реального применения этих видов тестирования:

5

- При разработке мобильного приложения интернет-магазина проводится функциональное тестирование, чтобы проверить корректность работы фильтров товаров, корзины покупок и

атакам хакеров (нефункциональный подход).

Ключевые аспекты:

- Функциональное тестирование: проверка отдельных функций программы против заданных требований;
- Метод черного ящика: способ функционального тестирования без изучения внутренней структуры программы;
- Нефункциональное тестирование: оценка характеристик, не связанных напрямую с функциями, таких как скорость работы, удобство пользования и защищенность данных.

Пример из практики:

Допустим, мы разрабатываем онлайн-сервис бронирования авиабилетов. Для него необходимо провести следующие виды тестов:

- Провести функциональные тесты каждого модуля (поиск рейсов, бронирование билетов);
- Выполнить нефункциональные тесты: протестировать интерфейс сайта на удобство навигации, проверить работу сервиса при высокой загрузке серверов, удостовериться в надежности защиты персональных данных пользователей.

Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:05

Да 18:06 ✓✓

🔍 Понятие: Этапы жизненного цикла тестирования

✨ Объяснение простыми словами:

Представьте себе строительство дома. Сначала вы определяете, какой дом хотите построить, затем планируете его конструкцию, выбираете материалы, строите фундамент, стены, крышу и проверяете результат. Так же происходит и с программным обеспечением: прежде чем выпустить продукт пользователям

5



Сообщение...

серию шагов, известных как жизненный цикл тестирования. Эти шаги помогают обнаружить ошибки до выпуска продукта и улучшить качество конечной версии.



Практическое применение:

В реальной жизни эти этапы используются повсеместно.

Например:

- При разработке мобильного приложения важно проверить каждую функцию перед запуском;
- Крупные компании вроде Google или Facebook регулярно проводят тестирование своих продуктов, чтобы обеспечить стабильную работу сервисов;
- Банковские системы подвергаются строгому контролю, поскольку любые сбои могут привести к серьезным последствиям.



Ключевые аспекты:

- Планирование: Определение целей и задач тестирования;
- Тест-дизайн: Создание сценариев и случаев для проверки функционала;
- Выполнение тестов: Проведение запланированных проверок;
- Отчетность: Документирование обнаруженных ошибок и их решений.



Пример из практики:

Допустим, ваша команда разрабатывает интернет-магазин. После завершения разработки интерфейса и функций вам необходимо протестировать корзину покупок, оплату, регистрацию пользователя и другие важные элементы. Это позволит выявить возможные баги еще до выхода сайта в публичный доступ.

Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:06



да 18:06 ✓✓



Сообщение...



Да 18:06 ✓✓

Понятие: Тест-кейсы и чек-листы

Объяснение простыми словами:

Представьте себе, что вы готовитесь к поездке за границу. Вам нужен план действий («что взять», «как добраться до аэропорта» и пр.). Такой план может выглядеть как подробная инструкция – каждый шаг расписан детально (например, заказать такси, проверить документы). А иногда достаточно просто краткого списка дел («купить билет», «собрать вещи»). Так вот, тест-кейсы — это подробные инструкции для тестирования чего-либо (программы, сайта), а чек-листы — это короткие списки контрольных вопросов или пунктов.

Например, если нужно убедиться, что сайт работает корректно, тест-кейс скажет вам точно, какие действия совершить и какой результат ожидать, а чек-лист лишь перечислит общие области, требующие внимания («проверить форму регистрации», «посмотреть работоспособность кнопок»).

Практическое применение:

1. Функциональность: Перед запуском нового функционала необходимо убедиться, что все элементы работают правильно. Для этого используются тест-кейсы, содержащие пошаговые инструкции и ожидания.

2. Регрессионное тестирование: Когда вносятся изменения в существующий функционал, важно повторно проверить, не нарушили ли эти изменения ранее работавшие функции. Здесь помогут заранее подготовленные тест-кейсы и чек-листы.

3. UI-тестирование: Проверка удобства пользования интерфейсом тоже требует детального подхода через тест-кейсы, а затем быстрого беглого просмотра с помощью чек-листов перед релизом.

Ключевые аспекты:

- Четкость формулировок: Названия тест-кейсов должны быть



Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:06



да 18:06 ✓✓

🔍 Понятие: Ручное тестирование vs автоматическое тестирование

📌 Объяснение простыми словами:

Представь, что ты готовишь блюдо по рецепту. Если каждый раз внимательно читаешь рецепт и следуешь ему шаг за шагом — это ручное тестирование. Но если однажды записываешь все шаги в виде инструкции и поручаешь роботу готовить вместо тебя — это уже автоматическое тестирование.

✅ Практическое применение:

1. ✨ Компании используют ручное тестирование при разработке новых функций приложений, когда нужно проверить сложные сценарии взаимодействия пользователя.
2. 🚀 Автоматизированные тесты помогают быстро проверять стабильность приложения после внесения изменений в код.
3. ★ На крупных проектах часто применяют оба подхода: ручные тесты проверяют критичные моменты, а автоматика следит за стабильностью всего продукта.

💡 Ключевые аспекты:

- Ручное тестирование: более гибкое, но медленное; полезно для редких проверок или уникальных ситуаций.
- Автоматическое тестирование: ускоряет проверку повторяющихся действий, снижает вероятность ошибок.
- Комбинированный подход: оптимальное решение для большинства проектов, сочетает преимущества обоих методов.

5

👤 Пример из практики:

Разработчик выпустил новую версию сайта. Сначала команда

- Комбинированный подход: оптимальное решение для большинства проектов, сочетает преимущества обоих методов.



Пример из практики:

Разработчик выпустил новую версию сайта. Сначала команда провела ручное тестирование важных страниц, затем запустила набор автоматических тестов, которые проверили функциональность всех остальных элементов. Благодаря этому удалось найти мелкие проблемы до официального релиза.

Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:06

да 18:07 ✓✓

Понятие: Инструменты для автоматизации тестирования

Объяснение простыми словами:

Автоматизация тестирования — это процесс проверки качества продукта или приложения без участия человека, когда специальные программы выполняют задачи по проверке функционала вместо людей. Представьте, что вы каждый раз вручную вводите данные на сайте, нажимаете кнопки и сверяете результат с ожидаемым поведением системы. Автоматизированное тестирование делает эту работу за вас быстрее и точнее.

Практическое применение:

1. Проверка работоспособности веб-сайтов после каждого обновления кода при помощи инструмента Selenium.
2. Тестирование мобильного приложения сразу на нескольких устройствах через Appium.
3. Регулярные автоматические тесты интеграций между разными сервисами с использованием Jenkins.

5

Ключевые аспекты:

- *Selenium*: Популярный инструмент для автоматического

Ключевые аспекты:

- *Selenium*: Популярный инструмент для автоматического тестирования пользовательского интерфейса веб-приложений; работает с различными языками программирования.
- *Appium*: Аналогично Selenium, но ориентирован на мобильную платформу; поддерживает iOS и Android.
- *Jenkins*: Система непрерывной интеграции, позволяющая запускать тесты автоматически при каждом изменении кода.

Пример из практики:

Допустим, у вашей компании есть мобильное приложение, которое должно корректно работать на разных версиях операционной системы Android. С помощью Appium вы можете написать один сценарий теста, который будет запущен одновременно на десятках устройств разной конфигурации. Это значительно сокращает время на проверку и повышает качество конечного продукта.

Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:07



да 18:07 ✓✓

Понятие: Метрики качества программного обеспечения

Объяснение простыми словами:

Представь себе строительство дома. Чтобы понять, насколько хорошо построен дом, мы смотрим на разные показатели: прочность фундамента, надежность крыши, удобство планировки комнат. Так же и в разработке ПО: метрики позволяют оценить, насколько программа хороша, удобна и надежна. Это своего рода «индикаторы», которые дают разработчикам понимание того, что нужно улучшить.

5

Практическое применение:

- Оценка сложности кода: Цикломатическая сложность

Практическое применение:

- Оценка сложности кода: Цикломатическая сложность (Cyclomatic Complexity) помогает определить количество возможных путей выполнения программы. Чем выше этот показатель, тем сложнее поддерживать программу.
- Измерение удобства поддержки: Индекс поддерживаемости (Maintainability Index) оценивает легкость сопровождения и изменения кода. Он учитывает цикломатическую сложность, объем кода и другие факторы.
- Тестирование и отладка: Покрытие кода тестами (Code Coverage) показывает, какая часть программы проверена автоматическими тестами. Высокий процент покрытия снижает вероятность ошибок.

Ключевые аспекты:

- **Цикломатическая сложность** (количество ветвей в программе).
- **Поддерживаемость** (легкость внесения изменений без появления новых багов).
- **Покрытие тестов** (доля проверенных участков программы).

Пример из практики:

Разработчик получил задачу оптимизировать сложную функцию в приложении. Используя метрику цикломатической сложности, он увидел, что функция содержит слишком много условий и имеет высокую сложность. После рефакторинга и упрощения структуры кода, уровень сложности снизился, а поддерживаемость повысилась.

Перейти к следующему понятию? (да/нет)

5

This message was sent automatically with n8n

18:08



Да 18:08 ✓✓

5

Вы прошли данное обучение! Что хотите изучать дальше?